

Fractional Retirement Algorithm for Carbon Credits

Harold Robson

August 2, 2025

Abstract

A gas-optimised FIFO algorithm for retiring on chain fractional carbon credits.

1 Introduction

Carbon credits on registries such as Verra have a minimum denomination of 1, equating to 1 tonne of Carbon Dioxide. In our "bridging" pools, we set decimals to 2, enabling users to purchase the tokenised equivalent of 0.01 carbon credits. This could become a major issue for retirements off-chain. For example, if a user requested to retire a non integer amount of credits a , we would have no way to satisfy this and would have to resort to retiring $\lceil a \rceil$, in other words "rounding up". If we let the fractional part of a be defined by $\bar{a}$, we define the "loss" $\ell = \lceil a \rceil - a$. A uniform distribution of $\bar{a}$ on $[0, 1)$ would result in the following expected loss per retirement:

$$\mathbb{E}(\ell) = \mathbb{E}(\lceil a \rceil - a) = \mathbb{E}(\lfloor a \rfloor + 1 - (\lfloor a \rfloor + \bar{a})) = \mathbb{E}(1 - \bar{a}) = \frac{1}{2}$$

Losing on average half a credit per retirement is not economically viable (in real world terms this could be several pounds) so we propose the following retirement algorithm.

2 The Fractional Algorithm

2.1 Definitions

Define a retirement r as $r = (t, v) \in R = \mathbb{R} \times \mathbb{N}$. For the sake of notation let $\tau((t, v)) = t$ and $\nu(t, v) = v$. Let a "queue" Q be defined by $Q = \{r_1 \cdots r_n \mid \tau(r_i) \leq \tau(r_{i+1})\}$. Furthermore define a split function $s : R \times \mathbb{N} \rightarrow R^2$ by

$$s((t, v), n) = \begin{cases} ((t, n), (t, v - n)) & n \leq v \\ ((t, v), (t, 0)) & n > v \end{cases}$$

Define a "split" queue as $Q' = (Q \setminus r) \cup \{r_1, r_2\}$ where $s(r, n) = (r_1, r_2)$. Note that each queue has many potential "split" queues, so Q' is not determined by Q exclusively. Finally, given Q , we define a "bundle" B as a subset of any valid Q' satisfying $\sum_{r \in B} \nu(r) \equiv 0 \pmod{100}$.

2.2 Aims

An algorithm finding a maximal¹ B from Q could easily be mistaken as a variation of the subset problem, until we impose the condition $\forall b \in B, \nexists r \in Q \setminus B \text{ s.t. } \tau(r) < \tau(b)$, which can be thought of as "FIFO" (first in, first out). Our aim is to find an algorithm maximising B from a given Q but preserving "FIFO". Note that we have the split function at our disposal which greatly simplifies finding this algorithm.

¹by maximal we mean having the greatest sum $\sum_{r \in B} \nu(r)$

2.3 Defining the Algorithm

Firstly we find the sum of Q by $S_Q = \sum_{r \in Q} \nu(r)$. Define the residue $res = S_Q \bmod 100$, and find maximal i such that $a = \sum_{j=i}^{|Q|} \nu(r_j) > res$. Finally we create the split operation $\sigma : \mathcal{P}(R) \times \mathbb{N}^2 \rightarrow \mathcal{P}(R)$ with s by:

$$\sigma : (\{r_1, r_2, \dots, r_i, r_{i+1}, \dots, r_n\}, i, v) \mapsto \{r'_1, r'_2, \dots, r'_i, r'_{i+1}, r'_{i+2}, \dots, r'_n, r'_{n+1}\}$$

where:

$$\begin{cases} r'_j = r_j & j < i \\ s(r_i, v) = (r'_i, r'_{i+1}) \\ r'_j = r_{j-1} & j > i + 1 \end{cases}$$

It follows that $B = \{r'_j | r'_j \in \dot{Q}, j \leq i\}$, where $\dot{Q} = \sigma(Q, i, a - res)$. Note that $S_Q = \sum_{r \in Q} \nu(r)$ is invariant² under the application σ of the split operation to Q .

3 Proofs

3.1 Proof that B is maximal

Let B be defined by our algorithm for a given Q , and let Q' represent any valid split of Q (it is not necessarily the split defined above). By way of contradiction, suppose there exists $B' \subset Q' \cup Q$ such that $S_{B'} = \sum_{r \in B'} \nu(r) > \sum_{r \in B} \nu(r) = S_B$. Since both S_B and $S_{B'}$ are elements of the ideal $100\mathbb{Z}$, we can infer $(-1) \cdot B \in 100\mathbb{Z}$ by the multiplicative axiom of ideals, and thus $S_{B'} - S_B \in 100\mathbb{Z}$ by additive closure. Since we know that the sum of a split Q' is equal to the sum of Q ($S_Q = S_{Q'}$), we can infer that $S_{Q'} - S_{B'} = S_Q - S_{B'}$. Our claim (*) is that $S_Q - S_B = res$ (where res is the residue of $S_Q \pmod{100}$). We prove this as follows:

Let $\dot{Q} = \sigma(Q, i, v)$ where i and v are defined as in section 2.3. Note that $\dot{Q}$ is a specific example of a valid split of Q . By the invariance of the sum of a queue under splitting we obtain $S_Q - S_B = S_{\dot{Q}} - S_B$. Since $B \subset \dot{Q}$, we simplify this sum to $S_{\dot{Q}} - S_B = S_{\dot{Q} \setminus B}$. From section 2.3, B is defined as $B = \{r_j | r_j \in \dot{Q}, j \leq i\}$, so $\dot{Q} \setminus B = \{r_j | r_j \in \dot{Q}, j > i\}$. We then evaluated the sum $S_{\dot{Q} \setminus B}$:

$$S_{\dot{Q} \setminus B} = S_{\{r_j | r_j \in \dot{Q}, j > i\}} = \sum_{j=i+1}^{|\dot{Q}|} \nu(r'_j)$$

Note that since Q has been split once, $|\dot{Q}| = |Q| + 1$, and for $r'_j \in \dot{Q}$, if $j > i + 1$, then $r'_j = r_{j-1} \in Q$ which comes from the definition of σ in section 2.3. Hence the sum can be evaluated further:

$$S_{\dot{Q} \setminus B} = \left[\sum_{j=i+2}^{|Q|+1} \nu(r'_j) \right] + \nu(r'_{i+1}) = \left[\sum_{j=i+1}^{|Q|} \nu(r_j) \right] + \nu(r'_{i+1})$$

We now need to inspect the splitting function $s(r, n)$ closer. Since r'_{i+1} is the second element of the tuple, We will consider the cases 1) $a - res \leq \nu(r_i)$ and 2) $a - res > \nu(r_i)$ separately:

Case 1

Since $a - res \leq \nu(r_i) \implies s(r_i, a - res) = ((t, a - res), (t, v_i - (a - res)))$ we obtain:

$$S_{\dot{Q} \setminus B} = \left[\sum_{j=i+1}^{|Q|} \nu(r_j) \right] + \nu(r'_{i+1}) = \left[\sum_{j=i}^{|Q|} \nu(r_j) \right] - \nu(r_i) + (v_i - (a - res))$$

By definition, $v_i = \nu(r_i)$, and $a = \sum_{j=i}^{|Q|} \nu(r_j)$ so we are left with

$$S_{\dot{Q} \setminus B} = a - \nu(r_i) + (\nu(r_i) - (a - res)) = res$$

² $S_{Q'} = \sum_{j=1}^{|Q|+1} \nu(r'_j) = \nu(r'_i) + \nu(r'_{i+1}) + \sum_{j=1}^{i-1} \nu(r'_j) + \sum_{j=i+2}^{|Q|+1} \nu(r'_j) = \sum_{j=1}^{i-1} \nu(r_j) + \nu(n) + \nu(r_i - n) + \sum_{j=i+1}^{|Q|} \nu(r_j) = \sum_{j=1}^{|Q|} \nu(r_j) = S_Q$

so $S_Q - S_B = S_{\dot{Q} \setminus B} = res$ as required.

Case 2

Note that this case ($\nu(r_i) < a - res$) is in fact unreachable. First, note that since i is the largest index in Q such that $a = \sum_{j=i}^{|Q|} \nu(r_j) > res$, then $\sum_{j=i+1}^{|Q|} \nu(r_j) < res$ hence $a = \sum_{j=i}^{|Q|} \nu(r_j) = \left[\sum_{j=i+1}^{|Q|} \nu(r_j) \right] + \nu(r_i) < res + (a - res) < a$ so $a < a$ and we have a contradiction.

Finally, we return to the fact that $S_{B'} - S_B \in 100\mathbb{Z}$ for our final contradiction. From $B' \subset Q'$ we know $S_{B'} \leq S_{Q'} = S_Q = S_{\dot{Q}}$. Hence from (*) $S_{B'} - S_B < S_{\dot{Q}} - S_B = res$. Since $res < 100$ by definition, and $S_{B'} - S_B < res < 100$ yet $S_{B'} - S_B \in 100\mathbb{Z} \Leftrightarrow 100 | S_{B'} - S_B$ we have reached our final contradiction so there cannot exist B' such that $S_{B'} > S_B$ and B is maximal.

3.2 Proof that B preserves FIFO

This is somewhat simpler. Let us first recall the definitions of B, Q and the property FIFO:

$B = \{r'_j | r'_j \in \dot{Q}, j \leq i\}$ where $\dot{Q} = \sigma(Q, i, a - res)$ and i, a, res are defined in section 2.3,

$Q = \{r_1 \cdots r_n \mid \tau(r_i) \leq \tau(r_{i+1})\}$,

"B is FIFO" $\Leftrightarrow \forall b \in B, \nexists r \in Q \text{ s.t. } \tau(r) < \tau(b)$

By way of contradiction, suppose for some $b \in B$ there exists $r \in Q \setminus B$ such that $\tau(r) < \tau(b)$. By the definition of our splitting function σ , either $\tau(b) = \tau(r_k)$ where $k < i$, or:

$$\tau(b) = \tau(r'_i) = \tau(r_i)$$

which follows from the definition of the split s at index i . Note that Q is "ordered" ($\tau(r_i) \leq \tau(r_{i+1})$). Hence, $\tau(r) < \tau(b) \leq \tau(r_i)$ so the index of r which we label " j " (in other words $r = r_j$) is less than i . By the definition of B, B contains all r'_k where $k < i$. From the definition of our splitting operation σ , $r'_k = r_k$ when $k < i$, so we now know B contains all r_k where $k < i$. Since $j < i$ we can now infer $r = r_j$ **must** be an element of B so we have reached a contradiction with the assumption $r \in Q \setminus B$. Hence our assumption that B is not FIFO is incorrect so B must be FIFO.

3.3 Argument that the algorithm is gas optimised

This is a little less rigorous. Proving that there exists no other algorithm with lower time complexity than ours which preserves FIFO would be extremely tricky, so we instead show that our algorithm has a linear time complexity of $O(|Q|)$. This is significantly better than all solutions to the subset problem. Also note that our algorithm preserves FIFO which would not necessarily be the case were a solution to the subset problem employed.

Let $T(n)$ denote the number of elementary operations required by our algorithm on a queue Q with $|Q| = n$.

Step 1: Computing S_Q requires exactly n additions. Step 2: Computing the residue res requires 1 modular operation. Step 3: Finding the maximal index i requires at most n operations. Step 4: The split operation σ requires at most $2n + 1$ operations (copying elements and one split). Step 5: Selecting B requires constant time. Therefore: $T(n) \leq n + 1 + n + (2n + 1) + O(1) = 4n + O(1)$ Hence $T(n) \in O(n)$, proving linear time complexity.

4 Conclusion

We have shown that our algorithm has three useful properties. Not only does it preserve FIFO ordering, but also finds a valid bundle B in linear time and has a constant loss ℓ of 0 per retirement (at no point did it round $\nu(r)$ for any r). This makes it perfect for application in our carbon credit ledger, especially when paired with the extremely cheap gas costs of Etherlink L2.